

Designing Bault — from Figma to the live site

The complete process, 19 September 2026. Kit: shadcn design Plus (Figma). Site: Bault's own React + CSS — no Tailwind, no shadcn code.

At a glance

- **The look** is Bault + Lyra: Lyra's shapes, spacing and sections; Bault's colours and fonts. Set up once.
- **A Figma design never imports itself.** It becomes part of the site when it is rebuilt as code: look → `index.css`, text → `i18n.tsx`, structure → the page's `.tsx` file, behaviour → React.
- **You can do the look and the text alone.** Layouts and widgets are code — that part is fastest with me.
- **With me:** send a screenshot of a kit section and say where it goes — or send a batch and say "use these where they fit best".

1. The look: Bault + Lyra

Why not Lyra alone. Lyra is shadcn's style; alone, Bault would look like thousands of other sites. Bault's colours carry meaning (custody green = safe in the vault, brass = grades and prices, oxblood = warning), IBM Plex has a matching Hebrew face, and Frank Ruhl Libre and the mono serials are what make it read as a vault. Lyra brings what Bault lacks: professional layouts, spacing and components.

FROM THE KIT (LYRA)	KEPT FROM BAULT
Spacing, sizes, text sizes	Custody green; brass for grades and prices; oxblood for warnings
How buttons, inputs, tables and menus are built	IBM Plex Sans + Plex Sans Hebrew for text, Frank Ruhl Libre for headlines
The page sections and layouts	Serial numbers in IBM Plex Mono
Square, minimal corners	The dark photo stage for cards

The "something new" is what we invent on top — like the graded-case card on the landing page.

Made once, at the start — this is where "keep our colours and fonts" happens:

1. Figma → *Local variables* → **Style** → duplicate the **Lyra** mode → rename it **Bault**.
2. In the Bault mode change **only colours and fonts** (values: `shadcn design-guide.md` §4). Everything else stays Lyra.
3. Send me a screenshot of the variables panel (or a link, if Figma is connected). I copy the values into `apps/web/src/index.css` — the one file that holds every colour, size and font — and the whole site changes at once.

From then on: every section you drag in, set the frame to **Style: Bault**, and it already looks like the site.

2. What a design is made of — and where each part goes

A screen in Figma is four things. Only the first two are "fonts, colours, spacing and text".

LAYER	WHAT IT IS	WHERE IT LIVES IN BAULT	WHO CAN DO IT
Look	colours, fonts, spacing, corners, sizes	<code>apps/web/src/index.css</code> (named values like <code>--custody</code> , <code>--sp-4</code>)	you, alone
Text	every word, in Hebrew and English	<code>apps/web/src/shared/i18n.tsx</code>	you, alone
Structure	what is on the page, and in what order	the page's <code>.tsx</code> file	code (React)
Behaviour	what clicking does, where data comes from, empty / error / loading states	React logic + the Bault API	code

So: if your design only changes the **look** or the **text**, that's all you need. If it moves things, adds things, or has anything that *does* something — that's structure and behaviour, which is code.

3. Complex widgets — what you already have

Bault has no component library (its only dependency is React). It has its **own** components, built to its design system — about thirty, plus ~60 icons:

KIND	WHAT BAULT HAS
Actions	<code>Button</code> , <code>IconButton</code>
Forms	<code>Field</code> (label, hint, error), <code>MoneyField</code> , <code>PhotoInput</code> (camera/upload)
Containers	<code>Panel</code> , <code>DetailDrawer</code> (side sheet), <code>ConfirmationModal</code>
Navigation	<code>NavigationRail</code> , <code>PageHeader</code> , <code>ContextTabs</code> + <code>TabPanel</code> , <code>UserMenu</code> , <code>LanguageSwitcher</code> , <code>ThemeToggle</code> , <code>NotificationBell</code>
Data	the <code>data-table</code> register, <code>StatusBadge</code> and status pills, <code>MetricCard</code> , bar charts, <code>timeline</code> , <code>stepper</code> / <code>steps</code> , chips, <code>Amount</code> , <code>Serial</code> , <code>Barcode</code> + label printing, card photo thumbnail and viewer
Feedback	<code>EmptyState</code> , <code>ErrorState</code> , <code>SuccessNote</code> , skeleton loaders
Icons	~60 in <code>shared/ui/icons.tsx</code>

Most live in `apps/web/src/shared/ui/` (`primitives.tsx`, `DetailDrawer.tsx`, `NavigationRail.tsx`, `PageHeader.tsx`, `PhotoInput.tsx`, `Serial.tsx`, `icons.tsx`).

What Bault does *not* have yet: a searchable dropdown (combobox), a date picker, a carousel / gallery slider, a range slider, tooltips and popovers, toast notifications, an accordion, pagination, multi-select filters, drag-and-drop.

When a kit design uses one of those, in this order:

1. **Reuse the closest Bault component.** A kit "card grid" is usually a Bault panel or register; a kit "modal" is `ConfirmationModal` or the `DetailDrawer`.
2. **Build it in React.** Simple ones — accordion, tooltip, carousel, pagination — are a few dozen lines each, styled with Bault's values.
3. **Add a proven library for the hard ones** — anything with complex keyboard and screen-reader rules (combobox, date picker, multi-select). The usual choice is **Radix UI primitives**: unstyled, accessible, and what shadcn itself is built on, so it matches the kit's behaviour exactly while Bault's CSS provides the look.

For dates, **react-day-picker**. Every library is code the site ships and must keep updated — it's a decision per widget, not a default.

A widget is only finished when it: works by keyboard, works right-to-left in Hebrew, works in dark mode, and works on a phone.

4. Route A — you design in Figma and put it in yourself

The whole process

1. **Design it in Figma**, with the frame set to *Style: Bault*.
2. **Read the values:** click each element in Figma. The right panel shows its size, spacing, colour and font.
3. **Open the page's file** in the project, e.g. `apps/web/src/areas/customer/marketing/LandingPage.tsx`.
4. **Write the structure** in React: the sections, headings, text and buttons, using Bault's existing pieces (`Button`, `Serial`, tables).
5. **Write the look** in `apps/web/src/index.css`, using Bault's named values (`var(--sp-4)`, `var(--ink)`) that match what Figma showed.
6. **Add the text** to `apps/web/src/shared/i18n.tsx`, in Hebrew and English.
7. **Look at it:** run `pnpm dev:web`, open `http://localhost:5173`, and compare it with Figma until it matches.
8. **Check it:** `pnpm --filter @bault/web typecheck · node scripts/design-lint.mjs · pnpm test:ux`.
9. **Save it:** add a note to `DIVE1.md`, then commit and push to GitHub.

Figma is the drawing; the code is the building. Steps 3–6 are the real work, and they need React and CSS.

Bault already has structure — so most changes are edits

YOUR DESIGN...	WHAT YOU CHANGE
Same layout, new look (colours, spacing, fonts, corners)	only <code>index.css</code> — the structure stays; the most common case
Small layout changes (move the price above the title, add a line, remove a section)	edit the existing tags in the page's <code>.tsx</code> : move one up, add one, delete one
Something new (a section or screen that doesn't exist)	write new structure

What "write the structure" means

Describing in code **what is on the page, and in what order** — not how it looks. A Figma box with a title, a sentence and a button:

```
<section className="landing-section">
  <h2>{t('landing.close.title')}</h2>           {/* the title    */}
  <p>{t('landing.close.body')}</p>               {/* the sentence */}
  <Button variant="gold" onClick={goSignUp}>      {/* the button   */}
    {t('landing.close.cta')}
  </Button>
</section>
```

- `<section>` is the box, `<h2>` a heading, `<p>` a paragraph, `<Button>` Bault's button.

- `t('...')` pulls the words from `i18n.tsx`, so they show in English or Hebrew.
- `className` is the name `index.css` uses to give the box its look.

Your Figma layers panel is already this list: frame → title → text → button. Writing the structure is turning those layers into tags, in the same order.

Honestly: the look and the text you can do alone today. Structure and behaviour are real React work — design it, handle the look and text, and hand the rest to me.

5. Route B — you design with me from their screenshots

Option 1 — you choose where each section goes.

1. Browse the kit in Figma (*Pro Blocks* → Landing / Application / E-commerce).
2. Screenshot a section you like.
3. Send it with one line: *"This one, for the marketplace listing page. Keep their layout, use our photos and prices."*
4. I rebuild it in Bault — Bault + Lyra look, real data, both languages, phone and dark mode — send screenshots, and it goes live on the tunnel.
5. You react in plain words ("tighter", "bigger photo", "the other one"). When you're happy I finish, test and write the DIVE1 note.

Option 2 — I choose.

1. Screenshot a batch you like (5–20), or give me Figma access to the whole kit.
2. Say: *"Use these wherever you think they fit best."*
3. I send a plan first — which section goes on which screen, and why ("E-commerce 4 for the store", "Application 12 for the vault list").
4. You approve or change it; I build one screen at a time, and you review each before the next.

Screenshot tips: capture the whole section; include its name from the Figma layers panel; screenshot its states (empty, hover, error) if it has them.

What I need besides the picture — or I choose something sensible and tell you: the real text if you have it; where the data comes from; what each button does; the empty / error / loading states; a phone screenshot if the phone layout differs; real card photos only, never invented items.

Why the file still matters. A screenshot is enough for me to build a screen *you* designed, because I rebuild with Bault's own parts rather than copying pixels. But the kit's value is its design decisions — layouts, spacing, how a checkout or a filter works — and I can only use what I can see.

6. Figma: which plan, and without it

FIGMA PLAN	I CAN READ YOUR DESIGNS	I CAN DRAW IN YOUR FILE	COST
Starter (free)	yes — 20 requests a month	no	\$0
Professional, Dev seat	yes — 200 a day	no	\$12/month
Professional, Full seat	yes — 200 a day	yes	\$16/month

- Figma: *"You need a Full seat to write to Figma files with agents."* One month is enough for a design push; drop back to free after.
 - Free is enough for **you** to use the kit by hand, and for screenshots to me.
 - **Without Figma:** I design straight in the site, or as mockup pages, from your words, sketches or screenshots of sites you like. You lose moving things yourself — and the kit, which is a Figma file.
-

7. Suggested start

1. Import the kit into Figma (free plan is fine).
2. Make the **Bault** style mode (Lyra + our colours and fonts) and send me a screenshot of the variables.
3. Screenshot 3–5 of their Landing sections you like.

I apply the new look to the whole site, then rebuild the landing page from their sections — and we go from there.

Details: `shadcn-design-guide.md` (sign-in, what's in the kit, the colour values, setup for designing with me) ·

`DESIGN.md` (the rules) · `apps/web/src/index.css` (the real values).